Robust Alpha Discovery

项目学习路线手册

从读懂研究设计，到能够独立接入真实数据、运行严格 OOS
并解释多重检验结果

建议周期 6 周	8 个学习阶段	17 项测试护栏

这份手册的目标

不是让你机械读完所有源码，而是让你在每个阶段都完成一次“读懂 - 手算 - 修改 - 测试 - 解释”的闭环。学完后，你应该能够说明每一个研究选择为什么存在，以及它防止了哪一种偏差。

1. 如何使用这份手册

当前项目状态

工程框架、合成数据全链路、20 个信号、OOS、成本、多重检验、regime、图表和 17 项测试已经完成。真实 A 股研究尚未开始，因此学习时可以放心重复运行合成数据，不会污染最终 holdout。

推荐的三遍学习法

遍数	目标	你要做的事	完成标志
第一遍	建立地图	只读 research_protocol、base.yaml 和 pipeline.py，画出数据流。	能用 3 分钟讲清项目从输入到报告的全过程。
第二遍	验证关键定义	按阶段读模块，手算小样本，再运行对应测试。	能解释 target、IC、HAC、turnover 和 maxT 的计算。
第三遍	主动改写	每阶段修改一个参数或实现，先写测试，再比较输出变化。	不依赖提示也能定位结果变化来自哪一层。

总阅读顺序

阶段	主题	核心内容
0	基础补齐	Python / pandas / 基础统计 / 收益时点
1	研究地图	协议、配置、总管线
2	数据与时点	schema、loader、universe、align
3	信号与标准化	specs、features、cross_section
4	统计评价	IC、factor return、HAC、metrics
5	组合与成本	weights、turnover、costs
6	严格验证	walk-forward、多重检验、bootstrap、regime
7	工程复现	pipeline、scripts、reporting
8	测试与真实数据闸门	tests、协议冻结、holdout

建议先完成阶段 0-4，再开始看组合收益。否则很容易把组合曲线当成研究起点，而忽略数据对齐和统计推断。

阶段 0

补齐最低前置知识 建议 3-5 天

结果：能读懂 DataFrame 操作、收益公式、相关系数和 t-stat。

这一阶段要学什么

- Python：函数、类型、模块导入、异常、Path、命令行参数。
- pandas：groupby、shift、rolling、merge、pivot，以及 MultiIndex 思维。
- 统计：均值、标准差、标准误、相关、回归斜率、p-value、第一类错误。
- 金融时点：信号形成、下单、成交、持有和退出必须是不同概念。

按这个顺序读文件

顺序	文件	带着什么问题读
1	README.md	项目究竟研究什么，而不是用了什么模型？
2	research_protocol.md	哪些规则必须在看结果前冻结？
3	data/README.md	一行数据代表谁、在什么时点可用？

动手练习

□ 用 5 行价格手算 close-to-close、next-open-to-open 两种收益。

□ 用 pandas 写出单只股票的 shift(1)、shift(-1) 和 rolling(5)。

□ 解释为什么 random train/test split 不适合这个项目。

通过标准

- 看到 shift 正负号时，能准确说出它引用过去还是未来。
- 能区别“特征可以用到 t”与“标签可以引用未来价格”。
- 能说明显著性不等于经济可交易性。

常见误区

直接从收益曲线开始学；把统计公式背下来却不检查时点；认为 p-value 小就一定有 Alpha。

阶段 1

先建立项目地图 建议 2-3 天

结果：能说明配置如何驱动同一条可复现管线。

这一阶段要学什么

- 研究协议决定允许做什么，YAML 决定本次实验具体怎么做。
- pipeline.py 只负责编排，统计细节分散在各专业模块中。
- 自动生成文件不是事实来源；源码、配置和协议才是事实来源。

按这个顺序读文件

顺序	文件	带着什么问题读
1	research_protocol.md	H1-H4 如何被证伪？哪些规则仍是 DRAFT？
2	configs/base.yaml	20 个信号、成本、bootstrap 参数从哪里进入？
3	configs/experiment_v1.yaml	真实数据和 final holdout 为什么默认关闭？
4	src/robust_alpha/pipeline.py	run_pipeline 的五个 stop_after 阶段是什么？
5	scripts/01-05	为什么脚本很薄，而逻辑都在 package 中？

动手练习

□ 画一张自己的模块依赖图，标出数据、特征、评价、验证、报告。

□ 把 base.yaml 的 bootstrap_replications 改成 30，运行一次快速实验。

□ 比较 stop_after=screen 与 stop_after=report 分别生成哪些文件。

通过标准

- 能在不打开源码的情况下，仅看 YAML 预测本次会计算什么。
- 能找到任意输出表是由 pipeline.py 的哪一段生成。
- 知道 synthetic、validation、final holdout 三者不能混用。

常见误区

把参数散落到脚本里；修改生成 CSV 代替修改源码；在协议未冻结时打开 final holdout。

阶段 2 **数据层与无泄漏时点** 建议 4-6 天

结果：能从原始 OHLCV 构造可审计的 research panel。

这一阶段要学什么

- schema 是数据合同：字段完整不代表数据在历史时点真的可用。
- universe_member、tradable、history_count 共同决定 signal-date eligibility。
- target 是标签，不是特征；未来开盘价只能进入 target 构造。
- Parquet 日期过滤用于在读取层面保护 final holdout。

按这个顺序读文件

顺序	文件	带着什么问题读
1	data/schema.py	哪些错误会立即拒绝：重复键、负价格、OHLC 矛盾?
2	data/synthetic.py	合成数据怎样制造 serial structure、停牌和成分变化?
3	data/loader.py	CSV 与 Parquet 在 holdout 保护上有什么差异?
4	data/universe.py	eligible 是只用当前与过去信息生成的吗?
5	data/align.py	entry_open、exit_open 和 target_valid 如何对齐?

动手练习

□ 手工构造 1 只股票 5 天数据，逐行核对 t、t+1、t+2。

□ 制造重复 (date, asset)、low > high、缺失 open，观察 schema 报错。

□ 从 panel 随机抽 3 只股票 x 5 个日期，核对 target。

通过标准

- 能解释 eligible、target_valid、research_eligible 的差别。
- 能证明任何 feature row 的最大输入日期不超过 signal date。
- 知道真实数据还需补充 ST、涨跌停、复权和上市期限规则。

常见误区

用今天的指数成分回填历史；把未来复权信息直接写回过去；看到 t 日收盘后仍假设按 t 日收盘成交。

阶段 3 **候选信号与横截面标准化** 建议 5-7 天

结果：能新增一个信号，并保证公式、方向和时点可测试。

这一阶段要学什么

- specs.py 保存 ex-ante 定义；family 文件保存公式；core.py 负责派发。
- rolling 和 shift 必须在单只 asset 内完成。
- 横截面 transform 必须按 date 分组，并先排除 ineligible observations。
- 同一信号尝试很多 scaling 方法也属于 hidden multiple testing。

按这个顺序读文件

顺序	文件	带着什么问题读
1	features/specs.py	20 个信号的 family、参数和预期方向是什么？
2	features/core.py	名称如何被派发到不同公式函数？
3	momentum/reversal.py	mom_5 和 rev_5 为什么符号相反？
4	volatility/liquidity/price.py	比例型特征如何降低价格水平影响？
5	preprocessing/cross_section.py	rank_gaussian 为什么能降低厚尾影响？

动手练习

□ 手算一只股票的 mom_5、rev_5、neg_vol_10。

□ 新增一个只使用过去信息的实验信号，并先在 specs.py 标注 post-hoc。

□ 改变第二天全部股票数值，验证第一天的 z__signal 不变。

通过标准

- 能解释 raw signal 与 z__signal 的区别。
- 未来输入被大幅修改时，历史信号逐位不变。
- 真实结果与预期方向相反时，不会悄悄翻转信号。

常见误区

跨股票混用 rolling；用全样本均值标准化；看到结果后改变方向仍称预注册。

阶段 4

统计评价：IC、Factor Return 与 HAC 建议 5-7 天

结果：能同时回答“是否可预测”和“统计证据有多稳”。

这一阶段要学什么

- 每日 Spearman IC 衡量横截面排序，而不是线性价格预测。
- 每日横截面回归斜率形成 factor-return 时间序列。
- 普通 t-stat 假设过强；HAC 对自相关与异方差更稳健。
- ICIR、Sharpe 和 p-value 回答不同问题，不能互相替代。

按这个顺序读文件

顺序	文件	带着什么问题读
1	evaluation/ic.py	每日 IC 如何处理最小股票数和常数信号?
2	evaluation/factor_return.py	横截面 beta 怎样转成时间序列检验?
3	evaluation/metrics.py	年化收益、波动、Sharpe、回撤如何计算?
4	tables/ic_summary.csv	20 个信号有哪些正负结果?
5	tables/factor_return_summary.csv	naive 与 HAC 结论是否一致?

动手练习

□ 构造 10 只股票的完美单调 signal/target，验证 IC=1。

□ 构造已知 slope=0.01 的横截面，检查回归能否恢复。

□ 对比同一 signal 的 IC t-stat、naive factor t-stat 和 HAC t-stat。

通过标准

- 能解释为何 Spearman 比 Pearson 更适合厚尾横截面。
- 能说清 HAC 修正的是标准误，而不是改变平均收益。
- 能报告负 IC 和不显著结果，而不是只保留胜者。

常见误区

把 IC 当成收益率；只看普通 t-stat；忽略候选信号数量就解释单个 p-value。

阶段 5 **组合、换手与交易成本** 建议 4-5 天

结果：能把预测边际转换为透明、可压力测试的组合。

这一阶段要学什么

- long-short 用于诊断纯信号；long-only 更接近 A 股现实约束。
- 权重必须在 signal-date 横截面确定，不能根据未来收益选股票。
- 严格 turnover 应与上一期收益漂移后的权重比较。
- 成本不是一个精确常数，而是 0/10/20/30 bps 的敏感性分析。

按这个顺序读文件

顺序	文件	带着什么问题读
1	portfolio/weights.py	top/bottom quantile 如何处理排序、并列和归一化?
2	portfolio/turnover.py	为什么先漂移权重，再计算调仓量?
3	portfolio/costs.py	bps 如何通过 turnover 转成收益扣减?
4	tables/portfolio_performance.csv	哪些信号对成本最敏感?
5	figures/06_portfolio_costs.png	成本如何改变累计收益路径?

动手练习

□ 用 10 只股票手工构造 top 20% / bottom 20% 权重。

□ 让两个资产收益不同，手算漂移前后 turnover。

□ 选择一个高换手信号，比较 0/10/20/30 bps 的 Sharpe。

通过标准

- long-short 多头腿和空头腿分别归一化，净暴露接近 0。
- 成本增加时净收益按 turnover 单调下降。
- 能区分统计显著、gross profitability 和 net implementability。

常见误区

直接比较目标权重忽略漂移；只报告零成本曲线；把诊断性空头当成现实可交易。

阶段 6 **OOS、多重检验与非平稳性** 建议 7-10 天

结果：能判断“最好结果”是否只是搜索和 regime 的产物。

这一阶段要学什么

- walk-forward 只在 train 选择，下一年 test 只评价不修改。
- Bonferroni 控制 FWER；BH 控制 FDR；二者回答的问题不同。
- joint block bootstrap 同时抽取所有信号和连续时间块，保留联合依赖。
- regime 必须少而可解释，并只使用 signal-date 可得信息。

按这个顺序读文件

顺序	文件	带着什么问题读
1	validation/walk_forward.py	top-k 是在哪些年份上选择的?
2	validation/multiple_testing.py	Bonferroni 与 BH 的 adjusted p 如何计算?
3	validation/bootstrap.py	为何先中心化，再联合抽样日期 blocks?
4	validation/regimes.py	volatility/trend/dispersion 是否使用未来阈值?
5	validation/nulls.py	循环位移如何破坏 alignment 但保留结构?

动手练习

□ 构造一个 train 强、test 反转的信号，验证选择规则不会补救它。

□ 给定 4 个 p-value，手算 Bonferroni 和 BH。

□ 固定 seed 运行两次 bootstrap，确认 max-stat 分布完全一致。

□ 解释为什么单独 bootstrap 每个信号会破坏相关结构。

通过标准

- 能解释 naive 8、BH 3、Bonferroni 1、maxT 1 的方法差异，但不把合成结果当市场结论。
- 能说明 global p-value 与单信号 adjusted p-value 的区别。
- 能判断 regime 差异是否稳定，而不是寻找最高 Sharpe 的环境。

常见误区

边看 OOS 边调窗口；把相关信号当独立检验；定义很多 regime 后只展示最好一个。

阶段 7 **工程复现、脚本与报告** 建议 3-4 天

结果：能从一份 YAML 一键生成全部研究资产。

这一阶段要学什么

- 编号脚本是用户入口，package module 是可测试的业务逻辑。
- 所有随机过程必须由配置 seed 控制。
- CSV、PNG、Markdown 和 manifest 必须来自同一配置。
- 报告需要明确 synthetic、exploratory、validation、holdout 的证据等级。

按这个顺序读文件

顺序	文件	带着什么问题读
1	scripts/01-05	每个 stop_after 对应哪个研究里程碑?
2	pipeline.py	每个 artifact 在何处创建、如何命名?
3	reporting.py	图表、报告和 manifest 怎样保持一致?
4	run_manifest.json	本次运行环境、seed 和输出如何追溯?
5	IMPLEMENTATION_STATUS.md	哪些完成，哪些故意没有完成?

动手练习

□ 删除 generated demo 后，用同一配置重新生成并比较表格。

□ 为一个新表增加 pipeline 输出、report 引用和 smoke test。

□ 检查每张图能否仅凭配置和源码重现。

通过标准

- 能说清源码、配置、生成物三者谁是 source of truth。
- 不同 seed 只影响随机模块；同一 seed 结果完全一致。
- 生成报告不会手工改 CSV 或图片中的数字。

常见误区

把 notebook 当最终生产管线；报告数字手工复制；生成物覆盖后无法追溯配置。

阶段 8 **测试驱动与真实数据闸门** 建议 4-6 天

结果：能安全接入真实数据，同时保持 final holdout 未触碰。

这一阶段要学什么

- 测试不是事后证明代码正确，而是预先定义不可违反的研究约束。
- 先运行最小单元测试，再运行 smoke，再运行完整 demo。
- 真实数据接入前必须冻结 point-in-time universe、复权、交易状态和样本切分。
- final holdout 只能在 FROZEN V1 后运行一次。

按这个顺序读文件

顺序	文件	带着什么问题读
1	test_schema_and_alignment.py	数据与 holdout 的硬边界是什么?
2	test_no_lookahead.py	未来数据变化为何不能影响历史 feature?
3	test_features_and_preprocessing.py	公式和 date-local transform 怎样验证?
4	test_evaluation/portfolio/*.py	统计与经济实现分别有哪些已知答案?
5	test_multiple_testing/test_walk_forward.py	随机性和 train-only 如何保护?
6	test_pipeline_smoke.py	全链路最小验收是什么?

动手练习

□ 故意引入一次 off-by-one target 错误，确认 alignment test 能捕获。

□ 把 future OHLC 放大 100 倍，确认历史 feature 不变。

□ 写真实数据字段映射表，但先只跑 development，不读 holdout。

通过标准

- 17 项测试全部通过，且你能解释每项测试保护哪一种研究风险。
- research_protocol 中所有未冻结项都有明确答案。
- final_holdout_confirmed_untouched 在协议冻结前保持 false。

常见误区

测试只覆盖正常路径；修 bug 时删除失败测试；真实数据刚接入就直接跑 final holdout。

2. 六周学习与实施日历

下面按每周 5 个有效学习日设计。每天建议 60-120 分钟；如果基础较弱，把一周拆成两周即可。

周期	主题	主要任务	周末验收
第 1 周	研究地图 + 数据时点	读协议、配置、pipeline；手算 target；读 schema/universe/align。	能画数据流并通过 alignment 类测试。
第 2 周	信号 + 横截面处理	读 specs/core/families；手算 5 个信号；检查 date-local transform。	能新增一个 post-hoc 信号及测试。
第 3 周	IC + Factor Return	读 IC、回归、HAC；构造已知答案；解释统计输出。	能同时报告 predictive 与 inference 指标。
第 4 周	OOS + 组合 + 成本	读 walk-forward、weights、turnover、costs；做成本压力测试。	能解释 train-only 选择和净收益变化。
第 5 周	多重检验 + Regime	手算 Bonferroni/BH；读 joint bootstrap；检查三个 regime。	能解释 discovery count 和 global p-value。
第 6 周	复现 + 真实数据闸门	运行全链路；审查 17 项测试；填写协议；准备真实数据映射。	协议可冻结，但尚未打开 final holdout。

每天固定的 5 步闭环

步骤	动作
1. 预习	写下今天要回答的一个问题，不要先看结果。
2. 阅读	只读与问题直接相关的 1-3 个文件，标记输入、输出和时点。
3. 手算	用 1-10 行最小数据计算预期答案。
4. 运行	执行对应测试和最小脚本，比较程序与手算。
5. 复盘	记录错误、修正、仍不确定的假设和下一步。

进度原则

宁可少读一个文件，也要完成一次手算和测试。这个项目的价值来自可证伪、可复现和不泄漏，而不是阅读速度。

3. 每日学习记录模板

建议把下面模板复制到自己的学习日志。每学习一个模块写一份，不要只记“今天看了某文件”。

栏目	应记录什么
今日问题	例如：为什么 target 要用 t+1 open 到 t+2 open?
涉及文件	写出实际阅读路径和关键函数名。
输入 / 输出	列出 DataFrame 必需列、索引和新增列。
时间边界	这个函数最多能使用到哪个日期?
手算样本	记录最小数据与预期结果。
运行命令	记录测试或脚本命令。
观察结果	程序是否与手算一致？哪里不一致?
研究风险	它防止 look-ahead、survivorship、selection bias 还是其他风险?
下一步	只写一个最小可执行动作。

每阶段都要能口头回答的问题

□ 为什么不能 random split?

□ 你如何证明当前成分股没有回填历史?

□ 为什么 Spearman IC 比 Pearson 更适合这里?

□ HAC 修正了什么，没有修正什么?

□ 为什么 block bootstrap 必须联合抽取所有信号?

□ BH 控制的 FDR 与 Bonferroni 控制的 FWER 有什么区别?

□ turnover-heavy reversal 为什么可能在成本后失效?

□ final holdout 失败时，为什么不能继续调到成功?

判断是否真正学会

如果你只能复述代码做了什么，还不算学会；当你能解释为什么这样设计、替代方案会带来什么偏差，并用测试证明边界时，才算掌握。

4. 毕业标准与真实数据闸门

能力验收矩阵

维度	你应该会什么	可验证证据
数据	能审计字段、历史成分、交易状态、复权和 target alignment。	随机抽样能追溯到原始日期；alignment tests 通过。
信号	能新增可解释、只用过去数据的信号。	公式手算一致；未来扰动不改变历史值。
统计	能解释 IC、factor return、HAC、p-value 和多重检验。	能手算小例子并解释不同修正的发现数。
OOS	选择只发生在 train，test 只评价。	walk-forward 输出可追踪每个 test year 的训练窗口。
组合	能解释权重、漂移后 turnover 和成本。	gross/net 同时报告；成本敏感性完整。
工程	一份配置一键重现全部结果。	17 项测试通过；表、图、报告来自同一配置。
表达	能报告失败、局限和 post-hoc 结果。	90 秒介绍不夸大 synthetic 或 exploratory 证据。

打开真实数据前的最终清单

- □ 确认真实 universe 与 point-in-time membership 来源。
- □ 冻结复权、ST、停牌、涨跌停、IPO seasoning、缺失 open 规则。
- □ 确认 development / validation / final holdout 切分。
- □ 确认 final holdout 从未因本项目被查看；否则必须降级为 validation。
- □ 将 research_protocol 状态改为 FROZEN V1，并记录时间。
- □ 先运行 development，再运行 validation；final holdout 最后只运行一次。

> **最后一句话**
> 完成这个项目的标志，不是找到一条高 Sharpe 曲线，而是你能证明自己知道如何判断一条曲线是否值得相信。

建议下一步：从阶段 0 开始，用 test_schema_and_alignment.py 作为第一份可以逐行读懂的测试文件。